

Final Report

For Introduction to Generative AI and Large Language Models

Ivan Montejo de Garcini

Github Repo

Just in case I forget to attach a link to the repo I'll leave a copy [here](#).

Week 1: Introduction

Notes

We'll be focusing on transformers-based LLMs. Language models estimate the probability distribution of the sequence of words through the statistical patterns of languages to predict the next words (or tokens). They generate text based on an existing text. Different LMs may attach the task in different ways but all of them are based on predicting text. Some popular language modeling algorithms:

- N-gram models: Predict next word based on the sequence of previous words.
- Hidden markov models (HMM): Based on hidden states and transition probabilities. There's the probability of a state showing up and a probability of one state transitioning to another.
- Neural Language models: In our case feed forward NNs (Neural Networks). Predict the probability of a word given a context.

Word embeddings: Different algorithms like Word2Vec and Glove to capture the semantics and enable arithmetic operations between words.

Transformers: Uses self attention to give weights for the importance of certain words. Originally designed for machine translation. This architecture:

- Captures long range dependencies: Words build relations over distances, it's no longer necessary that the words are next to each other (As happened in Ngrams).
- Parallelize: This means faster processing but also inference.
- Great results: No explanation needed

Core components of the transformers architecture:

Attention means that every word in the input attends to every other word, which maps the relative importance of each word with the rest. Multi head attention means having different attention systems that focus on different aspects. Attention doesn't care about the order of the words. In the end it's based on $W \times W$. Thus the system receives the information of the position of each word in sequence and it's added to the embedding.

There are encoder and decoder models. The encoding-only models only get the encoder part of the model. This is used for representing the input and capturing contextual relationships. The decoder-only transformers are the generative models and they only use the decoder part. They specialize in

generating text, predicting words. The encoder-decoder models are used in machine translation, dialogue systems, summarization. These combine the capabilities of understanding and generating text.

Decoder only LLMs

These are the generative AI models. Here we find the GPT family, which were the first ones that were pre-trained. Pre-training assumes that the data distribution stays relatively similar, thus needing to fine tune for specific tasks or domains. GPT-3 could be considered the first LLM. Other decoder only models are LLaMA or Mistral. The architecture of the models might be designed following intuition and by trial and error.

Mixture of experts

This means that certain networks specialize in certain tasks, then when the model is used only part of it is used. It's more modular and you don't have to compute that much stuff.

Lab

This was the first lab so it was the most simple. The lab consisted on researching on tokenizers and answering three questions. The answers can be found inside the README.md file in the week-1 folder.

Learning outcomes

To be honest I was already familiar with the concept of tokenization but this lab made me question what its purpose was. Through this lab I have become more conscious of the tokenizations methods used for different purposes or languages. Some tokenization algorithms may work better on certain types of languages and the same can be said about the task. For example, more morphologically oriented systems may choose a specific algorithm or reduce vocabulary size when compared to other systems. This lab also helped me grasp some of the tokenization techniques that I have seen in the Machine Translation course, which I have taken simultaneously to this course.

Challenges

I had to search for the different tokenization algorithms and, while most of them were easy to grasp, some took more reflection.

Week 2: Using LLMs and prompting-based approaches

Notes

LLMs have been trained with massive amounts of data scraped from the internet and they usually come pre-trained. During this pre-training they learn, not only to predict the next word, but various association such as Style, tone and domain. GPT3 was one of the first to approach prompting in a few-shot way, claiming that prompting is a way to extract or retrieve from the model the capabilities. In the pre-training the model is not told that, for example, some language is not adequate, to be positive, polite and all that stuff. They have the ability to learn new tasks from some examples, few-shots. Prompting is providing instructions or inputs to a PTM (pretrained model) to

guide its output. Prompts can take many forms (Instructions, questions), so do outputs. Prompting guides the model to a desired output:

- Specificity: Leverage to a new emergent capabilities, such as writing poems. Previously you had to train a new model for this.
- Control
- Efficiency
- Creativity

Prompting techniques

Zero-shot prompting: Provide simple instructions or a question for the specific task but no example

Few-Shot prompting: Provide instructions or questions but also one or more examples of the desired output.

Chain of thought prompting: Enables complex reasoning capabilities through an immediate reasoning steps. Inject the reasoning logic to complete the task as part of the task. This way the provide intermediate steps or tokens, which usually result in a better output.

Prompt chaining: Combine multiple prompts together, breaking down a task into smaller subtasks.

Others:

Tree of thought can be used to encourage the LLM to break the problem down into smaller steps and explore multiple reasoning paths.

In context learning

It's a generalization of few-shot where the LLM is provided a context as a part of the prompt and ask the model to use it. Retrieval Augmented Generation (RAG) is a form of in-context learning. You give the information that you want the model to understand and use in the prompt. RAG adds on top of the search engine a LLM that looks into the relevant information retrieved by the search engines.

Prompt templates

A convenient way to create reusable templates that contain certain elements that can be modified while keeping the prompt fixed. Such as "Give a summary of {topic}"

Lab

For the task I asked Gemini to generate a law that forbids the emission of violent content in the Spanish public television written in formal and academical Spanish and no longer than 4000 characters. For this, three prompting techniques were tested: Zero-shot, One-shot (using the text in a webpage as reference) and, finally, chain of thought (still using the reference from the one-shot). I tried both the chatbot python script and the prompting jupyter notebook. I made most of this part in the notebook because the output suited better the task, as the output was written in markdown. I found all answers to be adequate and well written but with some minor differences:

1. Zero-Shot: The prompt was "Can you write a law that forbids the emission of violent content in the Spanish public television? The text should be no longer than 4000 characters. It should also be written in formal and academical Spanish.". The answer was very well written but with some caveats:
 - Lacking signatures of authorities: It is expected that published and enforced laws are signed by the authorities, which in the case of Spain most of the time include the king and the president. These were not present in this first answer.
 - Structure: Although the structure was adequate it did not completely resemble a true published law, as these have different chapters which contain the articles. The answer also lacks the date and number of the law.
2. Few-shot/One-Shot: The prompt was "Can you write a law that forbids the emission of violent content in the Spanish public television? The text should be no longer than 4000 characters. It should also be written in formal and academical Spanish. The structure should mimic the one found here: <https://www.boe.es/buscar/act.php?id=BOE-A-2015-3442>". The result using this prompting technique improved vastly, now containing the signatures of both the king and president (this one was a placeholder), date, law number and such. The structure also resembles the reference that was given, which was a real law. The only caveat is that the king's signature is from pre previous one, which makes it a bit outdated.
3. Chain of Thought: The prompt was "Can you write a law that forbids the emission of violent content in the Spanish public television? The text should be no longer than 4000 characters. It should also be written in formal and academical Spanish. The structure should mimic the one found here: <https://www.boe.es/buscar/act.php?id=BOE-A-2015-3442>. Expose some reasons that led to this legislation in the preamble. Some articles should refer to these reasons by explaining how they affect them. Replace al the placeholders with actual values, for example, the law should be written using today's date and signed by the current president and king of Spain. Make up the law number." The result is very similar to the previous one but with some differences. The structure does not completely resembles the reference and some protocol things are missing but in this one the placeholders are replaced with the current data for the king and president.

Overall I would say that the prompt technique that offered the best result was the second one, Few-Shot/One-Shot.

Regarding the second part of the exercise the in-context-learning notebook has been changed to display the table. Despite this, given that each table is a different prompt, the sometimes have different styles. The table works both in HTML and in Markdown.

Motivation

The main aim of this lab was to compare the result from querying an LLM using different techniques. Besides this, which was the core of this lab, I also wanted to see how these techniques work on a language other than English, in this case Spanish. I wanted to see also the level detail that an LLM will show when using the different prompting techniques. Specially in one-shot, I wanted to see how much the language model sticks to the reference of how much it deviates.

Learning outcomes

Overall I saw that the LLM was quite consistent and the quality of the answer was generally good. Still, using one-shot prompting or giving references to the model improves noticeably the outcome. Despite this, even with the references the model misses some details, which, in my case, were mostly related to past events not being updated. This may be happening because the model has been trained with data of several different past years and most of this data might not be very recent. Also because the most updated models are usually not available in the free tier.

Challenges

One technical challenge I found was setting up the LLM, as the specific model to be used was not correctly written so I was getting a runtime error.

Another challenge was to create the base prompt template so it contains only relevant information and also to limit the size of the output, as the texts I was getting in the beginning were too long and could not be compared easily.

Week 3: Evaluation of LLMs

Notes

Based on continuous testing along the development. This is monitoring, but not QA, although both use the same tests (wrongly). Usually masking.

Benchmarking and validation are kinda the same thing, you measure thing and see how well they run. Here usually comes the confusion matrix (true pos, false pos...) and metrics (precision, recall... also true positive and false positive rate).

Use case bridges benchmarking and validation. The second one measures how usable something is, while the first one measures the robustness, scalability and reliability. Evaluation is usually done through shared tasks and standardised test suites.

LLMs

They accept input in a flexible and forgiving way and should give a natural response in appropriate language for tasks (translation, summarization...). But what's the right thing when talking about the output of the model? Some criteria are

- Language: Is the language correct and adequate?
- Discourse: Does it flow well? does it remember?
- Social awareness: Is the output appropriate for the parties engaged and the conversation?
- Consistency: Do they answer to different outputs similarly
- Truthfulness
- Topical competence
- Compliance: Does it follow the regulations required?
- Common sense: Capability of reason

- Effective/efficiency: In terms of computational resources, time and such
- Creativity: Is the interaction fun and delightful?

Can we test this stuff automatically? It's possible for some of them, but also depends on the language.

Generally the more parameters better but also less efficient, theoretically, and the size of training increases. Our brains are still more efficient, they do better with less neurons and less training data.

One test is making it choose correct word orders (usually adjectives). But even humans have issues. Also disambiguation (from ambiguous sentences), Hellaswag (choosing from several continuation of a short story). There also tests for biases. Detecting human written and AI generated text is also a possible test.

The thing is world knowledge comes in the way of linguistic competence. The hellaswag should be adapted to the cultural values of the language. Is there a way to parametrise and encode stuff like the manifestation of politeness principle and the cultural values of a language? Could the test be designed in an antagonistic way?

The lab assignment is based on the question "Would an LLM give a different response to the same question in different languages? What if the country, age and such is specified?"

See the difference in different languages. Then the difference specifying the country. Then see whether they seem valid.

Lab

This is the content of the lab for the evaluation of LLMs, which can also be found in the file LabReport.md in the week-3 folder.

For this lab I programmatically queried the Mistral LLM with three sets of questions in Spanish, the code can be found in a notebook called questions.ipynb. The first set of questions is composed of the questions in English, the second set is composed of the basic questions translated into Spanish. The third set of questions are localized versions of the basic ones in Spanish. Finally, the third set is composed of a more localized and contextualized version of the third one.

As expected the answers to the general questions are very correct and they are acceptable in most cultures, using neutral dialect (although with some regional words such as using "saco" as "sweater"). As an observation, in the second question all prices are in Euros, which is an interesting choice given that most Spanish speaking countries do not use that currency. The prices also change from one answer to another.

- Is there a difference between responses in two languages? In how many questions?

In question #7 there was a noticeable cultural/religious difference just by changing the language of the question. The question refers to the traditional Easter meals and in the one written in Spanish the LLM writes about the *cuaresma* and how in some religious settings no meat is eaten during these days (in my experience this is not usual) so it mentions mostly non-meat based meals. In contrast, in the English written questions this is not mentioned and lamb dishes are listed as the traditional

easter meal. This might be due to religious differences, given that Spanish speaking countries are catholic, whilst English speaking countries lean more towards reformist branches of Christianity.

Question #11 also displays some differences, as tipping culture is different in USA (the answer seems to focus mostly in USA without mentioning other English speaking countries) and the Spanish speaking world, although the Spanish written question is answered with a more global response, mentioning USA, Canada and other countries in Europe.

I'd say there's difference in about 3-4 questions.

- Is there less difference if the questions are localised? In how many questions?

When we compare with the localized questions we usually see more specific information. I'd say about 33% of the answers comment on the Spanish values and/or traditions. It's funny how in the third question the localised version recommends *botellon*, which will be commented on the last question. Questions also include information regarding Spanish law, although some non localised questions in Spanish also assumed a Spanish localisation. These Spanish specific laws are mentioned in answers #2, #5, #6, #7, #12. The localised version also mention Spanish institutions such as *guardia civil*.

- Were there some entertaining responses or differences worth noting?

In the second question the LLM suggests *botellon* as a good event to meet new people, which strictly speaking is not a false statement. For context, *botellon* means that a group of people meet in a public place (a square, the street, a park, etc) generally to drink alcohol. This is generally frowned upon in Spain and it's, except for specific zones during local festivities, illegal. I don't think it's false information but suggesting *botellon* without commenting the risks (you can get a substantial fine) is not adequate.

There are some words that are translated literally from English such as "Azúcar libre" which sounds very funny because it means "free sugar" instead of "sugar free".

Motivation

The purpose of this lab was to compare the behavior of an LLM depending on the cultural hints and see if the LLM adapts to variables such as language, culture or specific locations and also whether they commit mistakes of say inadequate things because of these variables. I also chose a more localised version to see how specific the LLM can get when it comes to cultural/regional adaptation.

Learning outcomes

I learned how LLM can sometimes say things that are not correct or that are a bit misleading and how it's very hard to control these models and keep them from saying inadequate things given that they are, at their core, statistical systems.

I also learned how hard it's to evaluate LLMs. Comparing with traditional machine learning models evaluating this is not straightforward at all and depends heavily on the task. When it comes to traditional tasks such as NER or text classification evaluation is quite simple, but when it comes to generating language that should be informative and to adapt to the situations the evaluation becomes really hard, as there's no specific answer that is correct and there may be many correct answers. This makes distinguishing correct and wrong answers almost impossible as there are also answers that are inbetween.

Challenges

It's hard for me to evaluate the answers because I also lack knowledge to do it. This means that many times I had to search for information on a subject to see whether the LLM was right or not. Adding this to the difficulty when it comes to distinguishing correct and wrong answers and the process of evaluation becomes very time consuming. This kind of evaluation is also impossible to automate.

Regarding technical challenges I had issues when programmatically querying the LLM because of the limitations of the free tier, so I had to find some workarounds.

Week 4: Finetuning

Notes

Traditionally, ML models are trained for a specific use case and they perform poorly in other tasks. It is task-specific. The foundational (pre-trained) models for LLMs are trained using self-supervised learning using huge amounts of texts. This model can be retrained for a specific task, fine tune it. This fine-tuned model should be better at that task. There are different approaches to fine-tuning.

Prompting vs Fine tuning

When prompting the model gets the prompt and creates an output, but the parameters are not tweaked. When finetuning you get in the end an updated model.

Why fine tuning?

We want to **improve the performance** of the model in some way. Maybe we want to adapt the model to a different domain or a different distribution of data (maybe you have documents that are not public and are structured in a certain way). Finetuning also **requires less data** than training a model from scratch, rarely makes sense to train from scratch. It's also more **efficient**, as the knowledge is updated from the one already present.

A pre-trained model may be good at generating text but not at classifying stuff.

How

First we have to choose a pre-trained model and get the data to fine tune it. For this last part usually a dataset is prepared using data relevant to the specific task. This may well be the most time consuming part. Then we get into the iterative loop of (re)training and evaluating until the result is satisfactory.

Key parameters:

- Controls how much the models parameters are adjusted during each training step (epoch).
- Number of epochs: The number of times that the model sees the training set.

Fine tuning methods

Supervised fine tuning is the basic one and it's based on input-output pairs. Used for classification, Question answering and MT with labeled data. Simple and effective but it's data intensive. Also the end result is limited by the quality of the data.

Instruction tuning is an supervised method. The model is provided instructions and an expected results. It's a type of supervised finetuning.

Constructing the datasets is very time consuming and important, probably the most important part of the finetuning process is getting and processing the data.

Reinforcement learning from human feedback is a combination of the supervised one with reinforcement learning where the model gets punished or rewarded based on the human feedback of its outputs. This is good when the evaluation metrics are blurry and hard to define like creative stuff. Human feedback collection can be expensive or inefficient, requires careful design of the feedback and can be computationally expensive. Aligns the model behavior to the human preferences.

Direct preferences optimization is a process where, instead of having the reward and the finetuning separated, you have a dataset with the preferred output given previously (chosen and rejected). Maybe more sample efficient than the previous one but it's a very modern approach and the result can depend on quality and diversity of the human choices.

Parameter efficient fine tuning

We want to reduce the cost while maintaining performance. This is not easy, as there's a huge amount of parameters. The idea is not to require to load all the parameters into the GPU, instead you train them in a modular way. Note that finetuning generally makes the model worse for general purposes and better for the specictally trained ones.

LoRA (Low Rank Adaptation)

Prefix turning: Adds trainable prefix to the input sequence, guiding adaptation without losing core parameters

Quantization addresses the huge amount of storage required for parameters by reducing the precision of the model. The idea is to use fewer bits instead of the standard 32 bit float. This means reduced model size and Faster inference but performance might be lost (but minimal unless precision is a huge factor).

Lab

In this lab I fine-tuned the model *Qwen3* with 4 billion parameters using a dataset composed of multiple choice questions and its answers in Spanish taken from the exams required to work in the Spanish public administration.

Motivation

I wanted to see how a multilingual model reacts to a very specific dataset. I also wanted to check how “multilingual” is really a multilingual model, given that it can be assumed that most of the data it was trained on was in English.

Learning outcomes

I realized that model usually have a bias towards the biggest language they were trained on (the one with the most data). Despite this the answer was correct and the model worked well.

Challenges

Given that a multilanguage model behavior is defined on training and the distribution of the training data may be unequal when it comes to the source language of the data there might be errors when it comes to answering, meaning that, even though the expected language of the answer was Spanish, the model answered in English.

There were also some technical challenges when making the lab. The main one was that, I guess, not all models have the same tokenization and they might have some different way of referring to specific tokens. In this case it was the BOS (Beginning of sentence) and EOS (End of sentence) tokens that had a different name, so it had to be changed in the code.

Week 5: RAG

Notes

RAG (Retrieval augmented generation) is a two phase process based on the retrieval of a document on a database, an evidence for the query, and you give it to the model as context. There as a retriever of documents and a generatos. The first one is non parametric, generally returns the top k. The accuracy depends on the correctness of the documents.

Agentic RAG involves decision making and such, mimicking humans, memory... The memory also works by remembering (storing) details about the user and the past queries though a set of parameters that are helpful to adapt the info to the user. This has access to APIs, can pre-process, can validate retrieved information.

Agentic RAG

It's supposed to have some level of autonomy. The agent is the LLM and has a defined role and task. The content of the chat can be stored (temporarily or not) to use that information. There's a layer that decides which model or module to load for the task.

Vector search

Using the encoding part of an LLM to get the vectors. It works on nearest neighbor, which is based on euclidean distance between two points (vectors). You can just use the distance but this gets crazy on a lot of data, to solve this it's also possible to divide the data into clusters. The most used algorithm is Hierarchical Navigable small Worlds.

Lab

I changed the pdf source document to the “*Manual de la Nueva Gramática de la lengua Española*”, which is the most widely accepted Spanish Grammar book written by the RAE (*Real Academia de la lengua Española*) to see how this search performs in a different language and in a specific domain.

The code in the “*00_simple_local_rag.ipynb*” notebook has been extensively tweaked to fit this text. The chunking system has been changed so that each chunk has some overlap with the previous and next chunk, thus giving the system some context window. It is expected that this overlap reduces the chance that important context is split across chunks and lost during retrieval, which should be applicable to every language. The tweaks have been made because each language might see different ratio of token/sentence. In this case Spanish usually has more tokens per sentence, as sentences tend to be longer than in English. Because of this the sentence extraction and chunking should be tweaked.

The embedding model has also been changed to fit this purpose. The embedding model chosen is [paraphrase-multilingual-mpnet-base-v2](#), which is a multilingual model that should support Spanish. Although the previous embedding model could also work for Spanish, using a multilingual model or one tailored for the specific language used might be desirable because embeddings work better if the model truly understands the language’s semantics, morphology, and distribution. For this to happen the embedding model must have been trained with data of such language (or at least similar).

Motivation

The idea is to test how the RAG pipeline would work in a different language and what should be changed in such pipeline to match or improve the performance when compared to English. Given that each language has different token distribution and structures the retrieval system should also be tailored to the language.

The same can be said about the embedding model, which should have been trained with data from the language used. The model has been changed to see whether this multilingual model performs better than the previous one. Despite this, the [previous model](#) did not state specifically that it was only for English. In fact, the previous model worked quite similarly.

Learning outcomes

This Lab has served me to grasp the concept of RAG, which is a concept I had heard but never knew what it was specifically. Through the lab I have learned how to create a system that can automatically search through a PDF file given a query using similarity search and which steps are necessary to make it work. While making the lab I have learned how to load a PDF and that the output must first be cleaned and chunked by, for example, replacing some characters or removing non useful chunks. After getting this clean list of chunks each of these can be embedded into a vector together with a query. Vector similarity can then be calculated between the query embedding and each of the vector embeddings to see which of the chunks can be the one most related to the query.

Challenges

This process can be very memory intensive and I ran out of GPU memory while performing the calculation with big vectors (depending of the query). In line with this, some things also took some time to execute, specifically the CPU operations, which I interrupted because I had access to a GPU. When reading and processing the PDF file I also had some issues with special Spanish specific characters such as the diacritics of the ñ. These characters would be printed as the encoding codes instead of the actual characters, but this issue did not persist in the output so it may be an issue with colab.

The performance of the RAG in Spanish also seems not to match with the English one. Although the results from the similarity search were related to the query the might not directly answer to it and it was less consistent, as the most similar one sometimes felt less similar than, for example, the third. So the top-k results somehow felt less consistent and sometimes the even felt like they were not related at all.

Week 6: Use cases and applications for LLMs

Notes

LLMs are evolving onto multimodality, managing language but also other kind of inputs such as images, video, audio, etc. The method for RAG of skipping OCR and recognizing a PDF as an image allows for faster processing, specially for images into the PDF. These multimodal LLMs also work by encoding each frame from a video as a separate image, but these images must also encode the timestamp, the sequence order is very important.

Use case are varied like chat systems for support or assistance for specific domains. This can be implemented to partially automate stuff or to retrieve more information if there's a issue for a support team. These models can also create synthetic data to get a starting point for a system, which should be replaced later by proper quality data.

Contrastive learning, links similar concepts together and dissimilates opposite or unrelated concepts. Used for text to image. Some models use image recognition for RAG instead of extracting the text with OCR.

When masking tokens in a transoformer the model doesn't really know the position of the token that has to be predicted, and when doing text ML that's fine. But that's not so fine for video and audio processing.

RAG can be done the opposite way by creating questions from a document, this is GAR.

Lab

In this lab I created a script that automatically queries the Mistral LLM to generate misspelled search queries found in a CSV file. For each of these 5 misspelled queries are generated. These misspelled queries are processed and saved into a dictionary and then into a json file (*misspelledQueries.json* inside the *week-6* folder). All the queries have been then used to compare the results in the DuckDuckGo search engine to see whether the misspelled queries return the same or similar results compared to normal queries.

In general, the misspelled queries showed similar results as the original one using DuckDuckGo. Usually the top 5 results were about 80% similar, although they would usually show up in a different order. Despite this, the results differed quite a bit after the top five, I'd say about 40-50% similarity. Another difference that I noticed is the behavior regarding other functions such as showing the map, youtube videos related, etc. Some of the misspelled queries activated different functions or in a different way. For example, using "*shortest route to Times Square from JFK Airport*" as a base query one of the misspelled versions did not activate the map/navigation function while the others did. Another example is "*petrol stations open now in downtown Los Angeles*". All misspelled versions activated the map/navigation function but most of them in different parts of the city. One of them even opened the map function in Spain (probably because of cookies because I am in Spain). Another example like this is "*walking directions to Eiffel Tower from Louvre Museum*", where all versions opened the map/navigation function but one of them mistook one of the places as placed it very far in the outskirts of Paris.

The fact that even when a query is heavily misspelled the search engine is still capable of recognizing the original query shows the systems present behind the scenes, which may use similarity search or some kind of corrector for unknown tokens (or both).

The synthetic data generated was pretty consistent. Most of the spelling mistakes seemed to be mistakes at writing (wrong key) or replacing with similar words (route → root). However, some of the spelling mistakes made little sense. For example, *direcshuns* is a spelling mistake that I would think is hard to make. This system could be improved by making a small set of rules that compiled the different categories of spelling mistakes (phonetic, omission, transposition, repetition) and using this small set of rules to few-shot the LLM. This small set of rules would, hopefully, give the language model an example from which the model would create new rules.

During this last Lab I also implemented the last bits of the previous Lab, notably the text generation though RAG and the page search with RAG. The model I used for text generation was [Llama-3.2-1B-Instruct](#) because it's a light model with a very modest amount of parameters that I can comfortably run in google Colab. The results from the text generation were notably good but with some mistakes on the reasoning. I asked to describe a linguistic phenomenon that's very common in Spanish (Elicit subject) and, while the description was quite accurate, the model illustrated the description with an example that wasn't the phenomenon that I asked to describe.

Motivation

The purpose of generating these misspelled queries was to create synthetic data which could be used for different purposes, in this case the purpose was to compare search engines but there are other tasks which may benefit from synthetic data. For example, in case there was a system that used the data from the users of such system the system would start with little to no data. In this case synthetic data could be used to kickstart the system.

Learning outcomes

One key learning outcome is understanding that search engines rely heavily on query normalization techniques such as spell correction and similarity matching to handle noisy or imperfect user input, improving robustness. The fact that similarity declines beyond the top five results shows that ranking stability decreases as relevance scores drop and auxiliary features, such as map/navigation or videos, seem to be the most affected ones. Small spelling variations can prevent feature

activation or cause incorrect geographic interpretations, highlighting the importance of entity recognition and disambiguation.

Regarding the synthetic data generation, the lab shows the strengths and weaknesses of using LLMs for synthetic data creation, given that some of the misspellings resembled human typing error but others felt unnatural.

Challenges

When querying the LLM for the misspelled versions of the query I was often exceeding the service tier capacity, but this was easily overcome with brute-force by just retrying in case of failure.

One challenge in the lab was generating realistic misspellings, as some LLM-produced errors did not reflect plausible human typing behavior. Also interpreting the results after the top ~5 seems difficult and subjective, as the results themselves may be similar but not exactly the same (same content but different url/site). Additionally, comparing results across misspelled queries required manual inspection, making the evaluation process time-consuming.